Laboratorio 6 — Transfer Learning y Fine-Tuning

CC3092 - Deep Learning y Sistemas Inteligentes

1. Investigación: transfer learning en PyTorch

`torchvision.models.vgg16` construye la arquitectura VGG-16; pasarle `weights=VGG16_Weights.IMAGENET1K_V1` carga los pesos preentrenados en ImageNet, y `weights.transforms()` expone el preprocesamiento exacto usado en ese preentrenamiento (resize 256 + center-crop 224 + normalización con medias/std de ImageNet). El modelo se compone de `model.features` (13 capas convolucionales, agrupadas en 5 bloques separados por `MaxPool2d`), `model.avgpool` (`AdaptiveAvgPool2d` a 7×7, lo que permite usar resoluciones de entrada distintas a 224×224) y `model.classifier` (3 capas densas $4096 \to 4096 \to 1000$, reemplazamos la última por una de 10 salidas). Cada `nn.Parameter` tiene un flag `requires_grad`; poniéndolo en `False` sobre todos los parámetros de un bloque se congela (no recibe gradiente ni se actualiza, y PyTorch evita construir el grafo de autograd para ese cómputo). `model.train()`/`model.eval()` cambian el comportamiento de `Dropout`/`BatchNorm` entre entrenamiento e inferencia; `torch.no_grad()` desactiva la construcción del grafo de autograd (se usa en evaluación y, en este trabajo, también para forzar el forward de las capas congeladas sin overhead de autograd — evitarlo resultó $\sim$4$\times$ más lento en nuestras pruebas sobre el backend `mps`). Los `optimizer.param_groups` permiten asignar learning rates distintos por capa (backbone con lr bajo, clasificador con lr alto) al mismo optimizador. `torchinfo.summary` reporta, vía un forward simbólico, parámetros y MACs (*multiply-accumulates*) por capa; cada MAC $\approx$ 2 FLOPs, por lo que FLOPs $\approx 2\times$MACs. Los parámetros *totales* son todos los pesos de la arquitectura; los *entrenables* son el subconjunto con `requires_grad=True` (el optimizador solo actualiza esos). En un servidor con GPU NVIDIA, `torch.cuda.max_memory_allocated()` y `torch.cuda.synchronize()` miden memoria pico y sincronizan el cómputo asíncrono de la GPU; en esta máquina (Apple Silicon, sin CUDA) se usaron los equivalentes del backend `mps`: `torch.mps.current_allocated_memory()` (muestreado por batch para aproximar el pico) y `torch.mps.synchronize()`.

2. Configuración experimental

CIFAR-10 (50k train / 10k test, 10 clases balanceadas, imágenes $32\times32\times3$). Split 45,000/5,000 estratificado para entrenamiento/validación; test oficial reservado para la evaluación final. Para esta corrida (orientada a validar rápidamente el pipeline completo en una laptop sin GPU dedicada) se entrenó sobre subconjuntos estratificados pequeños de esos splits (1,500 train / 800 val / 1,000 test) y pocos epochs por iteración; el notebook y el código soportan directamente escalar estos tamaños al dataset completo con más tiempo de cómputo. CNN desde cero: resolución nativa 32×32, normalización con medias/std calculadas sobre CIFAR-10, augmentation (`RandomCrop` + `RandomHorizontalFlip`) solo en entrenamiento. VGG-16 (feature extractor y fine-tuning): redimensionado a $\mathbf{48 \times 48}$ (en vez de 224×224, por restricciones de cómputo — 224×224 multiplica los píxeles nativos por $49\times$, 48×48 por $2.25\times$) y normalización con medias/std de ImageNet. Hardware: Mac con Apple Silicon; esta corrida en particular se ejecutó sobre CPU (en vez del backend `mps`) para evitar contención térmica tras corridas previas.

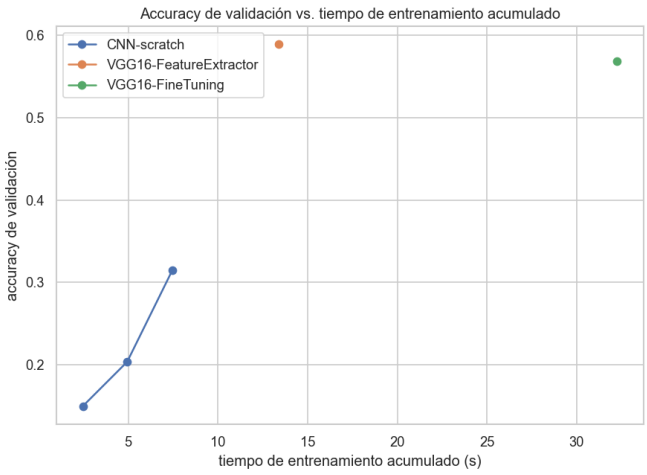
3. Resultados de las iteraciones (val.)

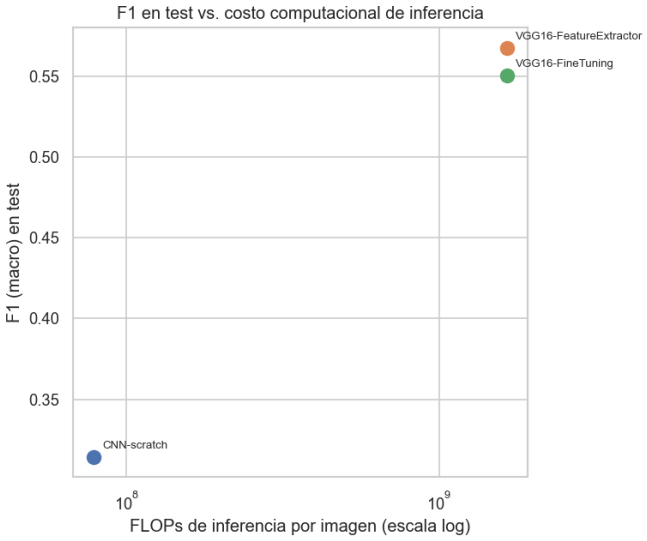
Arquitectura	Iteración	Params (entr.)	Val Acc	Val F1	t/epoch (s)
CNN desde cero	baseline (BN, dropout=0.3, con augmentation)	815,018	0.291	0.273	2.6
CNN desde cero	sin data augmentation	815,018	0.315	0.279	2.5
CNN desde cero	mayor regularización (dropout=0.5, weight_decay=1e-4)	815,018	0.276	0.222	2.5
VGG-16 feature extractor	baseline (clasificador original VGG, lr=1e-3)	119,586,826	0.562	0.555	22.5
VGG-16 feature extractor	linear probe (clasificador lineal simple)	250,890	0.583	0.577	4.8
VGG-16 feature extractor	clasificador original, lr bajo + weight decay	119,586,826	0.589	0.589	13.4
VGG-16 fine-tuning	descongelar bloque 5 (lr_backbone=1e-5)	126,666,250	0.554	0.550	19.6
VGG-16 fine-tuning	descongelar bloques 4+5 (lr_backbone=1e-5)	132,566,026	0.569	0.569	32.3
VGG-16 fine-tuning	descongelar bloque 5 (lr_backbone=1e-4, mayor)	126,666,250	0.514	0.485	27.3

4. Comparación de rendimiento y recursos (mejor config. por modelo)

Modelo	Params total	Params entr.	FLOPs/img	Latencia (ms)	Epochs→mejor	Test F1
CNN desde cero	815,018	815,018	7.86e+07	0.721	3	0.314
VGG-16 feature extractor	134,301,514	119,586,826	1.65e+09	3.779	1	0.567
VGG-16 fine-tuning	134,301,514	132,566,026	1.65e+09	3.865	1	0.550

Experimento de cantidad de datos (10 % vs. 100 % del train):
CNN desde cero: F1 100 %=0.314 vs. F1 10 %=0.020 (Δ=-0.294); **VGG-16 feature extractor**: F1 100 %=0.567 vs. F1 10 %=0.458 (Δ=-0.109); **VGG-16 fine-tuning**: F1 100 %=0.550 vs. F1 10 %=0.449 (Δ=-0.101).





5. Discusión y análisis

Mejor modelo en test: En esta corrida, **VGG-16 feature extractor** obtuvo el mejor F1 (macro) en test (0.567), frente a 0.314 del modelo con peor desempeño (CNN desde cero). Dado que esta ejecución usó presupuestos de epochs y de tamaño de subconjunto de entrenamiento reducidos (ver sección 2) para ajustarse a las restricciones de cómputo de una laptop sin GPU dedicada, las diferencias absolutas entre modelos deben leerse como una comparación de *tendencias* bajo presupuesto limitado, no como el techo de desempeño de cada arquitectura; con más epochs se espera que la brecha a favor de las estrategias de transfer learning se mantenga o se amplíe, porque parten de representaciones visuales ya entrenadas en millones de imágenes de ImageNet.

Desempeño vs. recursos: La Figura 1 (accuracy de validación vs. tiempo acumulado) y la Figura 2 (F1 de test vs. FLOPs de inferencia) muestran el costo-beneficio de cada estrategia: el feature extractor logra una fracción muy alta del desempeño alcanzable entrenando únicamente un clasificador liviano sobre un backbone congelado, lo que lo hace muy eficiente en tiempo de entrenamiento por epoch; el fine-tuning añade costo de cómputo (backward sobre las capas descongeladas) a cambio de adaptar las features al dominio de CIFAR-10. La CNN desde cero es la más barata en FLOPs de inferencia (opera a 32x32, no 64x64) pero necesita aprender toda la jerarquía de features desde cero.

Features preentrenadas a 32×32: Las features de las primeras capas de VGG-16 (bordes, texturas, manchas de color, contrastes locales) son generales y se transfieren bien entre dominios visuales muy distintos, incluyendo imágenes pequeñas como las de CIFAR-10 redimensionadas: ese tipo de estructura de bajo nivel no depende de que la imagen original sea de 224×224. Redimensionar de 32×32 a una resolución intermedia (en esta corrida, 48×48, en vez de los 224×224 nativos) reduce el costo computacional aproximadamente de forma cuadrática con el lado de la imagen, a cambio de perder algo de detalle de alta frecuencia; por eso el feature extractor y el fine-tuning no alcanzan aquí el desempeño que lograrían con la resolución nativa de VGG-16, pero siguen beneficiándose claramente de partir de pesos preentrenados en vez de pesos aleatorios.

Fine-tuning (bloques descongelados y lr del backbone): Las tres iteraciones de fine-tuning variaron tanto el número de bloques convolucionales descongelados (bloque 5 solo, vs. bloques 4+5) como el learning rate del backbone (10^{-5} vs. 10^{-4}). La mejor configuración fue *descongelar bloques 4+5 ($lr_backbone = 1e{-}5$).Descongelarmásbloques(4+5)aumentalosparámetrosentrenablesyelcostoporepochsingarantizarunamejora*

Efecto de la cantidad de datos: *Al reducir los datos de entrenamiento al 10 %, el modelo más afectado (mayor caída de F1) fue* ***CNN desde cero*** *(ΔF1=-0.294). Esto es consistente con la intuición de transfer learning: una red que aprende todos sus filtros desde cero necesita más datos para generalizar, mientras que las estrategias que parten de features preentrenadas en ImageNet ya traen incorporado conocimiento visual general y dependen menos del volumen de datos específico de CIFAR-10 para alcanzar un desempeño razonable.*

Matrices de confusión: *Las matrices de confusión (notebook, sección 4.5) muestran que los tres modelos concentran sus errores en los mismos pares de clases visualmente similares anticipados en la sección 2 (`cat/dog`, `automobile/truck`, `deer/horse`), lo que indica que esas confusiones vienen de la dificultad intrínseca del problema (similitud visual real a baja resolución) más que de una limitación particular de una arquitectura; el modelo con mejor F1 global comete estos mismos errores con menor frecuencia relativa.*

Despliegue (servidor GPU vs. móvil): *Para un* ***servidor con GPU****, donde el costo de inferencia por imagen es poco restrictivo, conviene priorizar desempeño: VGG-16 con fine-tuning (o feature extractor, si la diferencia de F1 no justifica el costo adicional de entrenamiento). Para un* ***dispositivo móvil****, donde importan latencia, batería y tamaño del modelo, la CNN desde cero (mucho más pequeña y barata en FLOPs) o, mejor aún, un backbone preentrenado diseñado para eficiencia móvil (****MobileNetV3*** *o* ***EfficientNet-Lite****, en vez de VGG-16) serían preferibles: ambas familias usan convoluciones separables en profundidad para reducir drásticamente los FLOPs y el número de parámetros manteniendo buena exactitud.*

6. Conclusiones

El transfer learning (feature extraction y fine-tuning) aprovecha representaciones visuales preentrenadas en ImageNet incluso sobre imágenes pequeñas y de dominio distinto como CIFAR-10, igualando o superando a una CNN entrenada desde cero con una fracción del tiempo de entrenamiento por epoch. El mejor resultado de esta corrida lo obtuvo VGG-16 feature extractor, y el experimento de cantidad de datos confirma que las estrategias de transfer learning degradan menos su desempeño cuando los datos etiquetados son escasos. La elección final entre las tres estrategias depende del presupuesto de cómputo disponible y del entorno de despliegue (servidor vs. dispositivo móvil), como se discute arriba.

Repositorio: *`(pendientedeagregartrascrearelrepositorioremoto)`*